CSC 316

Handout: D3, the Way We Use It

What this is. D3 is a big library; this course uses a small, consistent slice of it — **D3 for the math, Svelte for the rendering.** This handout is that slice. **Companion to Lab 3 onward.** Claude may use D3 for calculations in Lab 2, but you will study loading, scales, and data-to-mark mappings in Lab 3.

> **The course idiom:** we use D3 to *compute* (load data, parse dates, build scales) and let **Svelte** draw the SVG. We do **not** use `d3.select` / `.append` to manipulate the DOM. Knowing this up front makes the generated code make sense.

---

1. Loading and parsing data

`d3.csv` fetches a CSV over the dev server and returns an array of objects — one per row, every value a **string**.

```
import * as d3 from 'd3';

onMount(async () => {
  const raw = await d3.csv('/lax_passenger_traffic.csv');
  // raw[0] === { Terminal: "T1", Passenger_Count: "160413", ReportPeriod: "04/01/2021 12:00:00 AM", … }
});
```

Nothing is typed for you. Two conversions do almost all the work:

```
const rows = raw.map(d => ({
  ...d,
  Passenger_Count: +d.Passenger_Count,                          // string -> number
  date: d3.timeParse('%m/%d/%Y %I:%M:%S %p')(d.ReportPeriod)    // string -> Date
}));
```

`d3.timeParse` takes a **format string** and returns a parser function. The format describes the *input*:

Token	Means	Token	Means
`%Y`	4-digit year	`%m`	month, 01–12
`%y`	**2-digit year**	`%d`	day, 01–31
`%H`	hour, 00–23	`%I`	hour, 01–12
`%M`	minute	`%S`	second
`%p`	AM/PM	`%b`	month name, “Jan”

A parser returns `null` when the string doesn’t match — and a `null` date silently becomes a `NaN` pixel, which is why “all my points are stacked at the left edge” is usually a parse failure. Check one: `console.log(rows[0].date)` should print a Date, not `null`.

> **The two-digit-year trap.** `%y` maps 58 to **2058**, not 1958. Lab 4's Billboard data runs from 1958, so parsing its years with `%y` puts half the dataset a century into the future and the axis runs to 2058. If a time axis extends absurdly far right, this is the first thing to check. Use `%Y` with four-digit years wherever you can.

You can also pass a **row conversion function** as the second argument, which is tidier:

```
const rows = await d3.csv('/lax_passenger_traffic.csv', d => ({
  ...d,
  Passenger_Count: +d.Passenger_Count
}));
```

---

2. Scales

A **scale** is a function that maps a **domain** (values in your data) to a **range** (pixels, or colors). Every chart in this course goes through one.

```
const xScale = d3.scaleLinear()
  .domain([0, 700])      // data:   depth 0 to 700 km
  .range([0, 525]);      // pixels: left edge to right edge

xScale(0);      // 0
xScale(350);    // 262.5
xScale(700);    // 525
```

Domain in, range out. Every encoding in this course is a scale call.

`scaleLinear` — continuous numbers to continuous pixels. The default for quantitative data.

`scaleTime` — the same, but the domain is `Date` objects. Use it for any time axis; it also produces sensible tick marks (months, years) rather than raw timestamps.

```
const xScale = d3.scaleTime()
  .domain(d3.extent(rows, d => d.date))
  .range([0, innerW]);
```

`scaleBand` — for **categorical** data: it divides the range into evenly spaced bands, one per category, and handles the gaps.

```
const xScale = d3.scaleBand()
  .domain(['T1','T2','T3','T4','T5'])   // the categories, IN THE ORDER YOU WANT
  .range([0, innerW])
  .padding(0.2);                        // 20% of each slot becomes gap

xScale('T3');        // the left edge of T3's band
xScale.bandwidth();  // how wide each bar should be
```

Two things about `scaleBand` that matter: the **domain order is the display order** (so this is where you enforce a real rank rather than alphabetical), and `bandwidth()` is what you set a bar's `width` to.

`scaleSqrt` — for **size**. Because the eye reads a circle's **area**, mapping a value straight to radius exaggerates it enormously: double the radius is four times the area. `scaleSqrt` puts radius on a square-root footing so area tracks value:

```
const rScale = d3.scaleSqrt().domain([0, 8.8]).range([0, 30]);
```

And note the zeros: **area is *proportional* to value only when domain and range are both anchored at zero.** Start the domain at 6 instead and the areas are merely *ordered*, not proportional — the Lab 3 lesson.

Color scales — the same idea with colors as the range:

```
d3.scaleOrdinal().domain(types).range(d3.schemeTableau10);         // categorical
d3.scaleSequential(d3.interpolateViridis).domain([0, 700]);        // low -> high
d3.scaleDiverging(d3.interpolateRdBu).domain([-m, 0, m]);          // signed, zero in the middle
```

Which of those three you reach for is decided by your data's *type*, not by taste — that's the whole of Lab 7.

> **`.nice()` and `.clamp()`.** `.nice()` rounds a domain outward to friendly numbers, so an axis ends at 700 rather than 693.4. `.clamp(true)` stops values outside the domain producing out-of-range pixels. Both are small, both are worth knowing when an axis looks scruffy or a mark escapes the plot.

---

3. Helpers you'll see

Getting a domain from data — you rarely hard-code one:

```
d3.max(rows, d => d.Passenger_Count);     // largest value
d3.min(rows, d => d.Passenger_Count);     // smallest
d3.extent(rows, d => d.date);             // [min, max] — exactly a domain
```

`d3.extent` is the common one: it hands `.domain()` precisely the two-element array it wants.

Summary statistics:

```
d3.sum(rows, d => d.Passenger_Count);
d3.mean(rows, d => d.price);
d3.median(rows, d => d.price);      // better than mean on skewed data like price
```

Aggregation — `d3.rollup` groups rows *and* reduces each group in one call:

```
const byTerminal = d3.rollup(
  rows,                                    // the data
  v => d3.sum(v, d => d.Passenger_Count),  // how to reduce each group
  d => d.Terminal                          // how to group
);
// a Map: { "T1" => 151386083, "T4" => 161026428, … }
```

A `Map` isn't an array, so convert before rendering: `[...byTerminal]` gives `[key, value]` pairs, and `d3.rollups` (with an **s**) returns that array form directly.

`d3.group` is the same without the reduction — it keeps each group's rows intact, which is what you want for a multi-series line chart.

Sorting and formatting:

```
d3.ascending(a, b);  d3.descending(a, b);      // comparators for .sort()
d3.format(',')(1183357531);                    // "1,183,357,531"
d3.format('.1%')(0.85);                        // "85.0%"
d3.format('$,.0f')(3932.8);                    // "$3,933"
```

4. Axes

An axis reads a scale and produces tick marks and labels. This is the **one** place where the DOM-manipulating style of D3 is easiest, so we allow a small exception: generate the axis into a `<g>` that Svelte owns.

```
<script>
  import * as d3 from 'd3';

  let { xScale, innerH } = $props();
  let axisEl;                                  // a reference to the <g> below

  $effect(() => {
    d3.select(axisEl).call(d3.axisBottom(xScale));   // draw ticks into that <g>
  });
</script>

<g bind:this={axisEl} transform="translate(0,{innerH})" />
```

`bind:this` hands you the actual element; `$effect` re-runs whenever the scale changes, so the axis stays in sync. The four generators are `d3.axisBottom`, `axisLeft`, `axisTop`, `axisRight`; you'll use bottom and left.

Useful adjustments:

```
d3.axisBottom(xScale)
  .ticks(6)                          // roughly six ticks (a hint, not a promise)
  .tickFormat(d3.format('$,.0f'))    // format the labels
  .tickSizeOuter(0);                 // drop the little end caps
```

Axes never label themselves. The units line — "depth (km)", "passengers/month" — is a `<text>` element you add yourself. That's why "axis labeled with units" appears in every lab's checklist: the generator won't do it, so the agent frequently doesn't either.

5. Why not `d3.select`?

Classic D3 builds the DOM itself, with the data-join:

```
// the CLASSIC style — NOT what we do
svg.selectAll('circle')
   .data(quakes)
   .enter()
   .append('circle')
   .attr('cx', d => xScale(d.depth))
   .attr('cy', d => yScale(d.mag));
```

That works, and if you read D3 examples online you'll see it everywhere. We don't use it, for a specific reason: **Svelte already owns the DOM.** If D3 also creates and removes elements, two systems are editing the same tree with different ideas about what should be there — and they will disagree, usually as marks that fail to update or refuse to disappear.

Our version says the same thing declaratively:

```
{#each quakes as d}
  <circle cx={xScale(d.depth)} cy={yScale(d.mag)} />
{/each}
```

Same data, same scales, same result — and this version updates automatically when `quakes` changes, because it's ordinary reactive markup. There's no `.enter()` / `.exit()` lifecycle to manage, because Svelte handles the list.

So the division of labour is:

D3 does	Svelte does
Load and parse (`d3.csv` , `timeParse`)	Render marks (`{#each}`)
Compute scales (`scaleLinear` , `scaleBand` …)	Bind attributes (`cx={…}`)
Aggregate (`rollup` , `extent` , `sum`)	Re-render on change (`$derived`)
Generate axis ticks	Own the element tree

> **If Claude writes `d3.select(...).append(...)` for your marks**, that's a drift back to the classic idiom. Re-prompt: *"render the marks with `{#each}` in Svelte markup; use D3 for scales and axes only."* Your `CLAUDE.md` should say so once, and then you rarely have to repeat it.

Appendix — Scale cheatsheet

Scale	Use when	Example
`scaleLinear`	Continuous number → position	`.domain([0, 700]).range([0, 525])`
`scaleTime`	Dates → position	`.domain(d3.extent(rows, d => d.date)).range([0, w])`
`scaleBand`	Categories → evenly spaced slots	`.domain(terminals).range([0, w]).padding(0.2)`
`scalePoint`	Categories → single points (no width)	for a categorical axis on a line/dot chart
`scaleSqrt`	Number → **size** (area is what's read)	`.domain([0, 8.8]).range([0, 30])`
`scaleLog`	Number spanning orders of magnitude	`.domain([1, 100000])` — domain must not cross 0
`scaleOrdinal`	Unordered categories → distinct colors	`.range(d3.schemeTableau10)`
`scaleSequential`	Ordered/quantitative → one color ramp	`d3.interpolateViridis`
`scaleDiverging`	Signed, meaningful zero → two ramps	`.domain([-m, 0, m])`
`scaleQuantize`	Continuous → a few discrete bins	for a binned/stepped legend

Two rules:

- **The y-range is flipped:** `.range([height, 0])` , because SVG's y grows downward.
- **Size means area:** use `scaleSqrt` , and anchor domain *and* range at zero if you want area proportional to value.